

SCORM Gamified Authoring Tool — Implementation Plan

1. Context

The LMS currently **consumes** SCORM packages (upload, runtime, xAPI, reports — `lms-backend/src/routes/scorm.routes.ts`, `lms-backend/src/controllers/scorm.controller.ts`). It has no way to **produce** interactive, gamified course content in-platform. Authors today must use Articulate Storyline / Adobe Captivate externally.

Goal, per `data/api-docs/SCORM Gamified Content Tool Planning.docx`: a cloud-native, browser-based, slide-based authoring tool where instructional designers drag-and-drop text, images, video, audio, PDFs, and quizzes onto a canvas; add light variables/triggers for branching and scoring; preview; and export a SCORM 1.2-compliant PIF (ZIP). The exported ZIP is ingested through the LMS's existing SCORM material upload flow — **no schema coupling** to `bs_course` / `bs_module` / `bs_sub_module`.

Stack

- `lms-backend` (Node/Express + Drizzle + MySQL)
- `lms-worker` (Node, existing processor pattern)
- `lms-ui` (Vite + React + Tailwind + shadcn)
- New: `scorm-content-runtime/` package (Vite + React, ships inside every exported ZIP; runs inside any third-party LMS's iframe)
- Reused:
 - `learner-app/src/components/material-players/ScormPlayer.tsx` — the **LMS-side SCORM API host** (already implements `LMSInitialize/GetValue/SetValue/Commit/Finish` and proxies to `scorm.api.ts`). No changes required; used by the editor's Preview screen to host the just-built runtime exactly as learners will experience it. This is the "other side" of the SCORM protocol from `scorm-content-runtime/` — the existing player hosts content, the new package IS the content.
- New: `shared/authoring-schema/` package (Zod AST, shared by backend, worker, UI, player)

No Python runtime. `imsmanifest.xml` is templated with Handlebars/template literals, ZIP via `archiver`, LLM calls through the Anthropic SDK. See review-1 note: there is no justification for a second runtime here.

2. Source of truth: AST JSON + metadata tables

The v1 plan had a structural conflict — normalized per-component tables *and* a full-AST autosave endpoint. Choosing one: **the course AST is versioned JSON**. Relational tables hold only what needs to be queried, indexed, or joined across courses.

Data model (`lms-backend/src/db/schema/tables.ts`)

- `authored_courses` — id, org_id, owner_id, title, description, thumbnail_asset_id, status (`draft|published|archived`), current_draft_json (mutable working copy, JSON column), current_draft_etag (monotonic int, used for optimistic locking), current_version_id (FK → `authored_course_versions`, nullable), created_at, updated_at. **No FK to `bs_course/bs_module/bs_sub_module`.**
- `authored_course_versions` — id, course_id, version_number (monotonic per course), ast_json (immutable), published_by, published_at, notes. Every export references one of these. Never mutated after insert.
- `authored_assets` — id, org_id, content_hash (sha256, unique per org), kind (`image|video|audio|pdf`), storage_key, mime, size_bytes, duration_ms, width, height, poster_asset_id, original_filename, ai_alt_text, captions_vtt_key (nullable), scan_status (`pending|clean|infected`), created_by, created_at.
Content-addressable: uploading the same bytes returns the existing row (review-2 #1).
- `authored_course_asset_refs` — (course_id, asset_id) many-to-many, for quota/UX and so export can enumerate referenced assets without parsing the AST.
- `authored_exports` — id, course_id, version_id (FK required), job_id, status (`queued|running|success|failed`), package_url, size_bytes, manifest_hash, error, created_by, created_at.
- `authored_generations` — id, course_id, scope, prompt TEXT, model, options_json, pre_ast_snapshot_json (for undo), produced_artifact_json (the generated subtree or diff — populated only after full validation), status (`pending|running|ready|applied|reverted|failed`), tokens_in, tokens_out, cost_cents, error, created_by, created_at, applied_at.
- `authored_edit_locks` — (course_id PK, holder_user_id, expires_at) — short-TTL Redis-backed advisory lock to prevent last-write-wins collisions (review-2 #9). Plus an **If-Match: <etag>** check on every `PUT /ast`.

The AST schema itself (scenes → slides → layers → components → variables → triggers) lives in the shared Zod package below. MySQL stores it as a JSON column; queries use JSON path operators only for things we really can't avoid (e.g., "courses using asset X" — and we use `authored_course_asset_refs` for that instead).

MySQL triggers (whitepaper §3.3) are **not** v1.

3. Shared AST schema ([shared/authoring-schema/](#))

A new workspace package exposing a single Zod schema tree + TypeScript types (review-2 #3). Consumed by `lms-backend`, `lms-worker`, `lms-ui`, and `scorm-content-runtime` so there is one definition of what a valid course looks like. Any `PUT /ast`, AI-generated artifact, or player input is parsed through this schema; invalid payloads fail closed.

Top-level shape:

Course { id, title, meta, variables[], scenes[] }

Scene { id, title, slides[] }

Slide { id, title, background, durationMs, layers[], triggers[] }

Layer { id, name, visibleByDefault, components[] }

Component { id, type, frame{x,y,w,h,z}, properties (discriminated union by type) }

Variable { id, name, type, defaultValue, scope }

Trigger { id, event, condition?, actions[] } // ECA

Component types in v1: `text` | `image` | `video` | `audio` | `pdf` | `button` | `shape` | `hotspot` | `quiz_mcq` | `quiz_tf` | `quiz_fib` | `input`. Rich-text `text` uses a constrained schema (TipTap-compatible JSON) — no raw HTML.

4. Runtime state budget

SCORM 1.2 `suspend_data` has a hard practical limit of 4,096 characters. Review 1 correctly flagged this is a product constraint, not a footnote. v1 rules:

- The player's suspend payload is **only**: visited slide ids (bitset), current slide id, variable values, quiz answers (per-question option index only), total score. No free-text echoes, no interaction history objects.
- Variable names are minified to short IDs via a map stored inside the course version (not in `suspend_data`).
- Final step: `lz-string` compress → UTF-16 → base64.
- The editor renders a **live state-budget meter** that estimates compressed suspend size as the author adds variables, slides, and quizzes. Yellow > 2500 chars, red > 3500. An author cannot publish a course whose estimated `suspend_data` exceeds 3800 chars (safety margin). This meter is the author-facing "WCAG-style" constraint indicator recommended in review-1 #4.

- Free-text `input` components are limited to short responses (default max 100 chars) and, in v1, are **not** persisted across sessions — they reset on resume. This keeps the budget predictable.

SCORM 2004 fallback (64 KB suspend) is post-v1 (review-2 #4).

5. Backend (`lms-backend`)

New: `src/routes/authoring.routes.ts`,
`src/controllers/authoring.controller.ts`,
`src/services/authoring.service.ts`, `src/services/authoring-ai.service.ts`,
`src/db/repositories/authoring.repo.ts`. Register in `src/routes/index.ts`.

Endpoints (all JWT + RBAC, `authoring:manage` for CRUD, `authoring:ai` for generation, `authoring:publish` for version + export):

Authoring CRUD

- `POST /courses` — create draft (empty AST per shared schema).
- `GET /courses` / `GET /courses/:id` — list/detail. Detail returns `current_draft_json` + `etag`.
- `PATCH /courses/:id` — update metadata (title, description, thumbnail).
- `PUT /courses/:id/ast` — body is the full AST. Headers: `If-Match: <etag>`. Server parses via Zod, rejects on mismatch (409), bumps etag, stores JSON. Autosave is debounced client-side (~2 s).
- `POST /courses/:id/assets` — multipart upload. Server: sniff MIME (`file-type`), check against allowed v1 set (see §10), queue malware scan, compute sha256, return existing asset id if already stored in the org (content-addressable dedupe), otherwise persist + upload to GCS.
- `DELETE /assets/:id` — refuses if referenced by any course version.

Versioning + export

- `POST /courses/:id/publish` — snapshot current `current_draft_json` into a new `authored_course_versions` row, increment `version_number`, set `current_version_id`. Idempotent against unchanged drafts (hash compare).
- `POST /courses/:id/export` — body { `versionId` }. Enqueues `scorm-export` job. Returns export id.
- `GET /exports/:id` — status + signed download URL.
- `POST /exports/:id/send-to-lms` — **v1** convenience (review-1 #9): server-side forwards the produced ZIP bytes to the existing SCORM admin-upload endpoint and returns the resulting material id. No schema coupling; it's just an RPC call.

LLM-assisted authoring

All generations produce a **fully validated artifact** server-side before anything is returned to the UI (review-1 #6 — no incremental JSON Patch streaming). The UI streams **status/progress**, not low-level ops. Results are persisted to **authored_generations** for audit, undo, and budget tracking.

- `POST /courses/:id/generate/course` — `{ prompt, options: { slideCount?, audience?, tone?, includeQuiz?, language? } }`. Async via `authoring-ai.processor.ts`. Output: a full AST matching the Zod schema, ready to replace/merge into draft.
- `POST /courses/:id/generate/slide` — one new slide in context.
- `POST /courses/:id/generate/quiz` — **v1**. `{ sourceSlideIds?, topic?, count, types, difficulty }`. Output: one or more quiz slides with scoring variable wired, per-option feedback layers, and a summary slide with `set_score` + `complete_scorm` triggers at a configurable pass threshold.
- `POST /courses/:id/generate/rewrite` — `{ targetType, targetId, instruction }`. Output: a subtree to replace the target.
- `POST /courses/:id/generate/alt-text` — **v1 a11y** (review-2 #8). Fills `ai_alt_text` for one or many image assets.
- `POST /courses/:id/generate/captions` — **v1 a11y**. Auto-caption a video or narration audio; stores WebVTT at `captions_vtt_key`.
- `POST /courses/:id/generate/narration` — **v1**. `{ slideIds, voice, language, speed }`. Async TTS via pluggable provider (ElevenLabs / Azure Speech; per-org via `system-config`). Stores MP3 as an `authored_assets` row, attaches `on_slide_enter` → `play_media` trigger, auto-generates captions.
- `POST /courses/:id/generate/estimate` — **v1** (review-2 #6). Given a generation request, returns predicted tokens + audio seconds + cost cents. UI shows this before commit.
- `POST /courses/:id/generate/:genId/apply` — `{ mode: 'replace' | 'merge' }`. Server re-validates `produced_artifact_json` against Zod, then writes into `current_draft_json`, bumps etag. Returns new etag.
- `POST /courses/:id/generate/:genId/revert` — restores `pre_ast_snapshot_json`.

Image generation is **post-v1** (not a user-stated must-have; dedicated providers change fast; cost/safety tuning is its own project).

AI service implementation notes

- Provider: Anthropic Claude by default. Model + key per-org via `system-config`. Reuse any existing SDK wiring from `chatbot.routes.ts`.
- **Schema-constrained**: tool-use / JSON-mode with a stringified Zod JSON schema. Response is Zod-parsed server-side; on first validation failure, retry once with the validator's error message fed back; on second failure, mark `failed`. No half-valid artifact ever reaches the UI.
- **Prompt caching**: static system prompt + few-shot example courses live in `authoring-ai/prompts/`, marked with Anthropic cache breakpoints. Per-request task prompt is the only non-cached segment.
- **Long jobs** (`course`, `narration`) run in the new `authoring-ai.processor.ts` worker.
- **Safety**: moderation pass on prompt before call; strip keys from logs; per-org monthly token budget enforced via `system-config`; a generation whose estimated cost blows the remaining budget is rejected at `generate/estimate` time.

6. Worker (`lms-worker`)

Two new processors, following `src/processors/assignment.processor.ts`:

`scorm-export.processor.ts`

Input: `{ exportId }`. Steps:

1. Load `authored_exports` → `authored_course_versions.ast_json` (immutable — export is deterministic for a given version id).
2. Load `authored_course_asset_refs` for that course; filter to assets actually referenced by the AST.
3. Create a temp build dir. Copy the pre-built player bundle from `scorm-content-runtime/dist/` (checked-in at CI build time).
4. Stream each referenced asset from GCS into `build/assets/<sha256>.<ext>` (CAS = natural dedupe inside the ZIP too).
5. Rewrite asset references in a copy of the AST to `./assets/<sha256>.<ext>`; write to `build/course_data.json`.
6. Render `imsmanifest.xml` via `handlebars` template. Inputs: course title, version number, UUID SCO identifier, SCO `href="index.html"`, full file enumeration of the build dir (review-1 #8 — explicit file-enumeration rule).
7. Copy the four mandatory ADL schema files from the checked-in `scorm-content-runtime/scorm-schemas/` to the ZIP root: `adlcp_rootv1p2.xsd`, `imscp_rootv1p1p2.xsd`, `imsmd_rootv1p2p1.xsd`, `ims_xml.xsd`.

8. `archiver` → `course-<slug>-v<n>.zip`. Deterministic: fixed mtimes, sorted entry order, fixed compression level. Emits same ZIP bytes on re-run for the same version (reproducibility, review-1 #8).
9. Upload to GCS; compute manifest sha256; update `authored_exports` row.

`authoring-ai.processor.ts`

Input: `{ generationId }`. Loads the generation row, calls the AI service, validates, writes `produced_artifact_json` + status. SSE progress events go through an existing notification channel the UI can subscribe to.

7. SCORM content runtime (`scorm-content-runtime/`)

Scope clarification (important): this package is *not* a replacement for the existing `learner-app/src/components/material-players/ScormPlayer.tsx`. That file is the LMS-side **host** — it provides the SCORM API object for externally authored ZIPs to discover, and proxies the CMI calls to our backend. We keep and reuse it as-is.

`scorm-content-runtime/` is the opposite end of the protocol: the **content SPA** that goes *inside* every exported ZIP and gets launched by some third-party LMS's own host. It discovers whatever SCORM API that foreign LMS provides and writes completion/score/suspend data into it. Without this package, an exported ZIP has nothing to run.

A separate Vite + React package, built once in CI. Bundle is checked in / published as an artifact that `scorm-export.processor.ts` copies into every ZIP.

`scorm-content-runtime/`

`src/`

`main.tsx` ← bootstraps; discovers LMS SCORM API before mount

`App.tsx`

`scorm/`

`api-wrapper.ts` ← pipwerks-style discovery (window.parent up to 500; window.opener)

`cmi.ts` ← typed LMSInitialize/GetValue/SetValue/Commit/Finish

`suspend-data.ts` ← bitset + var-id minify + lz-string

`engine/`

trigger-engine.ts ← ECA evaluator (a11y + perf: run in Web Worker — review-2 #2)

variables.ts ← Redux slice

navigation.ts ← non-linear slide graph

components/ ← one renderer per component.type; a11y attrs baked in

a11y/ ← captions overlay, keyboard nav, focus ring, prefers-reduced-motion

index.html ← inlined via vite-plugin-singlefile

scorm-schemas/ ← ADL XSD/DTD files (committed)

manifest.template.hbs

vite.config.ts ← base: './', vite-plugin-singlefile

package.json ← "homepage": "."

Key properties:

- **Relative paths** (`base: './'`) + **single-file** HTML bundle to survive strict LMS iframes.
- **API discovery** walks `window.parent` up to 500 levels, falls back to `window.opener`.
- **suspend_data**: enforced state model from §4, compressed via `lz-string`, fail-loud if over 4096 chars (player sends a console warning + a best-effort `Commit`).
- **A11y**: WCAG 2.1 AA in v1 renderers (keyboard navigation, focus order, captions overlay, `prefers-reduced-motion` honored, alt-text piped from `ai_alt_text`).
- **Trigger engine in Web Worker**: UI thread is never blocked by condition evaluation, even in deeply branched courses.

8. Export contract

Before any worker/player code is written, a short spec doc lives at `scorm-content-runtime/EXPORT_CONTRACT.md` and defines, precisely (review-1 #8):

1. AST schema (link to the shared Zod types).
2. Asset resolution rules (CAS sha256 filenames, folder layout, URL rewrite exact regex).
3. Generated file layout of the ZIP root (required and optional files).
4. `imsmanifest.xml` generation rules (every attribute, every required element, where UUIDs come from).

5. SCORM CMI field mapping: which CMI element a trigger writes to for completion, score, suspend, exit, resume.
6. Determinism contract (same version id → byte-identical ZIP).

Both the worker and the player are written against this document.

9. Admin UI (`lms-ui`)

One new top-level app-sidebar entry, **Authoring**, under permission `authoring:manage`. It opens the course list. Everything described below lives **inside the editor page** (`AuthoringEditor.tsx`); the Tools palette is a panel within that page's own layout, not a new item on the main app sidebar.

- `pages/authoring/AuthoringList.tsx` — list + "New course" + **"Create with AI"** entry (prompt box, options, → async generate-course).
- `pages/authoring/AuthoringEditor.tsx` — a single full-screen editor page composed of these panels:
 - **Left panel (inside the editor page) — Tools palette** (the primary insertion surface). Grouped, searchable list of every v1 component type the author can place on a slide:
 - **Text**: heading, body, rich-text block, caption, quote.
 - **Media**: image, video, audio, PDF (sandboxed).
 - **Interactive**: button, hotspot, input (short text), shape (rect/ellipse/line).
 - **Quizzes**: MCQ, true/false, fill-in-the-blank.
 - **Structure**: layer, scene break, slide from template.
 - **Gamification**: variable, trigger preset (the curated set — see below).Each tool tile is a `dnd-kit` draggable source. Two insertion paths, both are always available on every tool:
 1. **Manual drag-and-drop** onto the canvas — the default. A drop target shows alignment guides and snapping.
 2. **"Generate with AI"** affordance (sparkle icon on the tool tile, also on right-click). Opens a compact prompt popover scoped to that tool ("what should this image show?", "what topic should this quiz cover?", "describe this slide"). Submitting calls the matching `generate/*` endpoint; the returned artifact is inserted at the cursor/drop position with the standard diff-accept overlay. Example flows: drag-drop an empty Image tile, then click its sparkle to generate the image; drag-drop a Quiz tile, fill the AI prompt "3 MCQs on payroll basics" → the quiz slide(s) land where it was dropped. This is the manual-vs-AI parity the user asked for — one palette, two insertion modes per tool.
 - Below the palette: a **scene/slide tree** for navigation.
 - **Center — canvas** (`dnd-kit`), the drop surface. Alignment guides, grid snapping, multi-select, keyboard nudge.

- **Right — inspector:** component props + triggers. The triggers panel ships in v1 with curated **preset templates** — "Navigate on click", "Branch on quiz result", "Show layer on hover", "Set variable on click", "Complete course when...". Each preset is also draggable from the Tools palette onto a component. An "Advanced" toggle reveals the raw ECA builder for power users. This resolves review-1 #5 without removing capability.
- **Top bar:** variables panel, state-budget meter, preview, publish, export, "Send to LMS" (calls `send-to-lms`).
- **Bottom — Refine bar:** prompt box scoped to current selection (course / slide / component). Calls `generate/rewrite` or `generate/slide`. Complements the per-tool sparkle: Refine bar edits what's already on the canvas; tool-sparkle inserts new content.
- **Diff overlay** on AI output: additions green, deletions red, modifications amber. Accept per-hunk or in bulk. Everything is hand-editable immediately afterward; any hand-edited element can be prompted again. This is the core manual-vs-AI loop the user asked for.
- **Generation history drawer** — list of `authored_generations`, with Undo.
- **Cost meter** — running session + org monthly.
- `pages/authoring/Preview.tsx` — reuses the existing `learner-app/src/components/material-players/ScormPlayer.tsx` as the host. The just-built `scorm-content-runtime/dist/index.html` (with the current course's `course_data.json` injected) is served via a temporary preview endpoint and loaded into the existing player component. This means preview runs through the same API host, backend proxy, and CMI plumbing the learner will use — no separate fake API path to drift out of sync.

State: Redux Toolkit slice `authoringSlice` holding the current AST + etag. Autosave debounced 2 s. Optimistic updates with rollback on 409 (version conflict) — UI shows "another author just saved; reload or overwrite".

New deps to add (check first): `@dnd-kit/core`, `@dnd-kit/sortable`, `@reduxjs/toolkit`, `react-redux`, `fast-json-patch` (diff visualization only, not transport), `eventsource-parser`, `dompurify` (rich-text sanitization at render).

10. Security & content

Explicit v1 rules (review-1 #7):

- **Allowed asset types:** `image/png`, `image/jpeg`, `image/webp`, `image/gif`, `video/mp4`, `video/webm`, `audio/mpeg`, `audio/wav`, `application/pdf`. **SVG is rejected** in v1 (script vector).

- **MIME sniffing** server-side via `file-type` — reject on mismatch with the declared MIME; do not trust the client.
- **Malware scanning**: a pluggable scan hook runs before the asset becomes usable. Scan pending → asset is `scan_status='pending'` and unusable in exports (`scorm-export` refuses to run on a course referencing unscanned assets).
- **Rich-text sanitization**: TipTap JSON → HTML conversion runs through `DOMPurify` on both save (server) and render (player + editor). No raw HTML input path exists.
- **PDFs**: rendered via `<iframe sandbox="allow-same-origin allow-scripts">` in the player, no form submission.
- **LLM retention**: prompts + outputs retained per the org's existing data retention config; generations older than N days can be purged via a scheduled task.
- **Prompt injection resistance**: AI service never concatenates raw user content into system prompts; all user text goes in structured user-role turns; tool-use JSON schema constrains what can come back.

11. Rollout phases

1. **Foundations**: shared Zod schema package, export contract doc, v1 allowed-asset list.
2. **Player** against fixture ASTs; validated against SCORM Cloud using hand-written JSON.
3. **Export worker** + `authored_course_versions` + `authored_exports`; reproducibility test; SCORM Cloud pass.
4. **Backend CRUD + asset pipeline** (CAS, scanning) + optimistic locking.
5. **Editor UI (non-AI)**: canvas, inspector, preset triggers, variables, state-budget meter, preview, publish, export, Send-to-LMS.
6. **AI suite v1**: course/slide generation, rewrite, quiz, TTS narration, alt-text, auto-captions, cost estimation, diff-accept, generation history + undo.
7. **Gamification polish**: advanced ECA mode, more presets, richer branching.
8. **Post-v1**: image generation, SCORM 2004 fallback, multi-SCO, semantic asset search, real-time collaboration.

12. Critical files

Read for context

- `lms-backend/src/routes/scorm.routes.ts`, `src/controllers/scorm.controller.ts` — patterns to reuse + the target for `send-to-lms`.
- `lms-backend/src/db/schema/tables.ts` — add new tables.
- `lms-backend/src/middleware/auth.middleware.ts` — reuse for RBAC.
- `lms-backend/src/storage/` — GCS adapter for assets + exports.
- `lms-worker/src/processors/assignment.processor.ts` — processor template.

- `lms-worker/src/registry.ts` — register new job types.
- `lms-ui/src/pages/learning/` — UX patterns + sidebar.

New files

- `shared/authoring-schema/` (new workspace package)
- `lms-backend/src/routes/authoring.routes.ts`
- `lms-backend/src/controllers/authoring.controller.ts`
- `lms-backend/src/services/authoring.service.ts`
- `lms-backend/src/services/authoring-ai.service.ts` (+ `authoring-ai/prompts/`)
- `lms-backend/src/db/repositories/authoring.repo.ts`
- `lms-backend/drizzle/migrations/NNNN_authored_*.sql`
- `lms-worker/src/processors/scorm-export.processor.ts`
- `lms-worker/src/processors/authoring-ai.processor.ts`
- `scorm-content-runtime/` (whole package + `EXPORT_CONTRACT.md`)
- `lms-ui/src/pages/authoring/*`
- `lms-ui/src/api/authoring.ts`
- `lms-ui/src/stores/authoringSlice.ts`

13. Verification

1. **Schema:** `npm run build` in `lms-backend`; drizzle migrate on dev DB.
2. **Shared schema:** unit tests covering invalid AST payload rejection (missing refs, wrong types, over-budget variable count).
3. **API:** `src/__tests__/authoring.test.ts` covering CRUD, etag conflict (409), asset dedupe, MIME rejection, SVG rejection, scan-blocking export.
4. **Versioning:** publishing an unchanged draft is idempotent; publishing a changed draft produces a new immutable row.
5. **Export worker:** fixture course → integration test asserts ZIP contains `imsmanifest.xml`, `index.html`, `course_data.json`, four XSDs, every referenced asset under `assets/<sha256>`. Run twice against same version id → assert byte-identical output.
6. **SCORM conformance:** upload exported ZIP to <https://cloud.scorm.com> test harness — must pass 1.2 validation; completion + score report correctly.
7. **Compatibility matrix** (review-1 #10): SCORM Cloud, our own LMS runtime, a strict-iframe fixture (`sandbox="allow-scripts"` only) — player must discover API or fail-loud in all three.
8. **suspend_data:** property-based test that generates 50 random courses, runs them through simulated completion, asserts compressed suspend never exceeds 4096 chars or the editor refused to publish.

9. **Player unit tests:** trigger engine (ECA evaluation), navigation (non-linear paths with backtracking), state serialization size, score/completion transitions.
10. **Security:** upload SVG → rejected; upload file with spoofed extension → rejected; attempt rich-text with `<script>` → sanitized on save and on render.
11. **A11y:** axe-core on player renderers; keyboard-only smoke test.
12. **UI end-to-end:** create course with 3 slides + 1 branching quiz + TTS on one slide; preview; publish; export; ingest via Send-to-LMS; complete as a learner; verify score + suspend/resume round-trip in the LMS.
13. **Build check** before push: `npm run build` in every workspace.

14. Out of scope (v1)

- Image generation (text-to-image).
- Real-time multi-author collaboration (WebSockets/CRDT).
- SCORM 2004 / xAPI export.
- Multi-SCO packages.
- Semantic vector search over assets.
- Mobile-responsive authoring canvas (player output IS responsive).